✨ Module 1

21 October 2026 21:08

🧠 Problem Solving Concepts

Problem Solving:

Problem Solving is the process of identifying a problem and finding a logical solution to achieve the desired goal.

Key Points:

1. **Understanding the Problem** – Identify the given information and required output.
2. **Planning the Solution** – Select an appropriate method to solve the problem.
3. **Executing the Solution** – Perform the planned steps systematically.
4. **Verifying the Result** – Check the correctness and accuracy of the solution.

💡 Computational Thinking Basics

Computational Thinking:

Computational Thinking is a problem-solving approach that breaks complex problems into manageable steps so solutions can be implemented efficiently by humans or computers.

Core Pillars:

1. **Decomposition** – Breaking a complex problem into smaller parts.
2. **Pattern Recognition** – Identifying similarities or recurring trends.
3. **Abstraction** – Focusing on important details while ignoring irrelevant ones.
4. **Algorithm Design** – Creating a step-by-step solution to solve the problem.

⚡ Algorithms

Algorithm:

An Algorithm is a finite sequence of well-defined instructions used to solve a problem.

Key Points:

1. **Step-by-Step Procedure** – Provides a systematic approach to solving a problem.
2. **Well Defined** – Each step is clear, precise, and unambiguous.
3. **Finite** – Must terminate after a limited number of steps.

3. **Finite** – Must terminate after a limited number of steps.
4. **Language Independent** – Can be implemented using any programming language.

📊 Flowcharts

Flowchart:

A Flowchart is a graphical representation of an algorithm using standard symbols and flow lines.

Key Points:

1. **Visual Representation** – Shows the sequence of operations graphically.
2. **Standard Symbols** – Uses symbols for process, input/output, decision, and terminal.
3. **Easy Understanding** – Simplifies complex logic and program flow.
4. **Helpful in Debugging** – Makes errors easier to identify and correct.

📝 Pseudocode

Pseudocode:

Pseudocode is an informal description of program logic written in simple English-like statements.

Key Points:

1. **Easy to Read** – Uses simple and understandable language.
2. **Language Independent** – Not based on any specific programming language.
3. **Represents Program Logic** – Clearly describes the sequence of operations.
4. **Easy Conversion to Code** – Can be directly translated into a program.

💻 Overview and Features of C Language

C Language:

C is a general-purpose, procedural programming language developed by Dennis Ritchie in 1972 at Bell Labs.

Key Features:

1. **Simple and Structured** – Easy to learn and organize programs.
2. **Portable** – Programs can run on different platforms without major changes.
3. **Fast and Efficient** – Provides high execution speed and performance.
4. **Rich Library Support** – Offers many built-in functions for programming tasks.

📜 Structure of a C Program

Structure of a C Program:

A C program consists of different sections arranged in a specific format.

Main Parts:

1. **Documentation Section** – Contains comments about the program.
2. **Header Files** – Includes required library files.
3. **main() Function** – Starting point of program execution.
4. **Program Body** – Contains statements and instructions.

⚙️ Compilation and Execution Process

Compilation and Execution Process:

It is the process of converting C source code into executable machine code.

Steps Involved:

1. **Source Code** – Program written by the programmer.
2. **Compilation** – Compiler translates source code into machine code.
3. **Executable File Creation** – Generates a runnable file.
4. **Execution** – Operating system runs the program and displays output.

🔤 Tokens

Tokens:

Tokens are the smallest individual units of a C program.

Types of Tokens:

1. **Keywords** – Reserved words with predefined meanings.
2. **Identifiers** – User-defined names for variables and functions.
3. **Constants** – Fixed values that do not change.
4. **Operators & Symbols** – Perform operations and separate statements.

🔑 Keywords

Keywords:

Keywords are reserved words in C that have predefined meanings and cannot be used as identifiers.

Key Points:

1. **Reserved Words** – Have special meaning in C.
2. **Fixed Purpose** – Their function cannot be changed.
3. **Cannot be Used as Names** – Not allowed for variables or functions.
4. **Examples** – int, float, if, else, return.

🏷️ Identifiers

Identifiers:

Identifiers are user-defined names used to identify variables, functions, arrays, and other program elements.

Key Points:

1. **Used for Naming** – Variables, functions, and arrays.
2. **Begin with Letter or Underscore (_) ** – Cannot start with a digit.
3. **Case Sensitive** – `sum` and `Sum` are different.
4. **Cannot be Keywords** – Reserved words cannot be used as identifiers.

📦 Variables

Variables:

Variables are named memory locations used to store data whose value can change during program execution.

Key Points:

1. **Stores Data** – Holds values during execution.
2. **Has a Data Type** – Such as int, float, char.
3. **Value Can Change** – Can be updated whenever required.
4. **Requires Declaration** – Must be declared before use.

🔒 Constants

Constants:

Constants are fixed values that remain unchanged throughout program execution.

Key Points:

1. **Fixed Value** – Cannot be modified once defined.
2. **Improves Readability** – Makes programs easier to understand.
3. **Used for Permanent Data** – Such as PI and tax rates.
4. **Types of Constants** – Integer, float, character, and string.

📊 Data Types

Data Types:

Data types specify the type of data a variable can store and the memory required.

Main Types:

1. **int** – Stores integer values.
2. **float** – Stores decimal values.
3. **char** – Stores single characters.
4. **double** – Stores large decimal values with higher precision.

➕ Operators

Operators:

Operators are special symbols used to perform operations on operands.

Types of Operators:

1. **Arithmetic Operators** – +, -, *, /, %
2. **Relational Operators** – ==, !=, >,
3. **Logical Operators** – &&, ||, !
4. **Assignment Operators** – =, +=, -=

⌨️ Input-Output Functions

Input-Output Functions:

Input-output functions are used to receive data from the user and display results on the screen.

Key Functions:

1. **printf()** – Displays output on the screen.
2. **scanf()** – Accepts input from the user.
3. **Formatted I/O Functions** – Uses format specifiers like %d, %f, %c.
4. **Facilitates User Interaction** – Enables communication between user and program.